

ШПАРГАЛКИ ПО ROS2 ДЛЯ РАЗРАБОТЧИКОВ

Quality of Service (QoS) В ROS2

Разработано в РТУ МИРЭА ИИИ КПУ
Грачевым А.В.
Зениной А.А.

ОГЛАВЛЕНИЕ

Что такое Quality of Service (QoS) в ROS 2?	3
Политики QoS: Строительные блоки	3
Стандартные профили QoS	5
Совместимость QoS (почему данные могут "не ехать")	6
Таблицы совместимости для ключевых политик	6
Reliability	6
Durability	6
Deadline и Lease Duration	7
Что происходит при несовместимости?	7
 QoS-события: Узнаём о проблемах	7
 Работа с QoS в коде (Python примеры)	7
Импорты	7
Явное создание профиля	8
Использование стандартных профилей	8
Создание "статического" publisher'a (как latching в ROS 1)	9
Подписка на QoS-события (Python)	9
 Диагностика: Как посмотреть QoS работающей системы	10
1. <code>ros2 topic info /topic_name --verbose</code>	10
2. Отладка совместимости	11
 Продвинутые темы и нюансы	11
JSON-представление QoS	11
 Финальная шпаргалка: Ты хочешь... Тебе нужно...	11
Обратная связь	12

Что такое Quality of Service (QoS) в ROS 2?

В ROS 1 коммуникация между узлами была преимущественно TCP (надёжно, как заказное письмо). ROS 2, работая поверх DDS, даёт возможность выбирать: быть соединению **надёжным как TCP** или **быстрым как UDP**, и это лишь малая часть настроек.

QoS (Quality of Service) — это набор политик, которые определяют, как именно данные передаются между издателем и подписчиком. Правильная настройка QoS позволяет системе быть одновременно и надёжной (для критичных команд управления), и производительной (для потоковых данных с камеры).

Политики QoS: Строительные блоки

Набор политик (**policies**) формирует **профиль QoS (profile)**. Вот полный список того, что можно настраивать.

Политика	Суть	Ключевые значения / Тип	Когда использовать
History (История)	Как долго хранить сообщения, если подписчик не успевает их обрабатывать?	Keep last (хранить последние N), Keep all (хранить все, пока хватит ресурсов).	Keep last — для большинства случаев. Keep all — когда нельзя потерять ни одного сообщения (например, запись данных для последующего анализа) .
Depth (Глубина)	Размер очереди сообщений (работает только с Keep last).	Целое число (например, 10).	Маленькая глубина (1) — если важна только последняя поза робота. Большая глубина (50+) — если подписчик "тормозит", но все данные нужны .
Reliability (Надёжность)	Гарантировать ли доставку каждого сообщения?	Reliable (надёжно, с подтверждениями), Best effort (максимум усилий, без гарантий).	Reliable — для команд управления и критичных данных. Best effort — для сенсоров (лазер, камера), где лучше получить свежий кадр, даже если старый потерян .
Durability (Стойкость)	Сохранять ли сообщения для подписчиков, которые подключатся позже?	Transient local (сохранять последние сообщения), Volatile (не сохранять) .	Transient local — аналог "latching" в ROS 1. Нужен, чтобы поздно подключившийся подписчик сразу узнал последнее состояние (например, карту местности) . Volatile — для всего остального.
Deadline (Дедлайн)	Ожидаемый максимальный интервал между сообщениями.	Длительность (Duration).	Если издатель обязан публиковать данные с определённой частотой, и нарушение этого критично.
Lifespan (Время жизни)	Максимальное время жизни сообщения. Если оно "провалилось" в очереди дольше — оно считается устаревшим и не доставляется.	Длительность (Duration).	Для данных реального времени: если сообщение с лазера пришло с задержкой в 1 секунду, оно уже бесполезно для локализации.
Liveliness (Активность)	Механизм проверки, жив ли издатель.	Automatic (система сама проверяет), Manual by topic (издатель должен	Для обнаружения "мёртвых" узлов в распределённых системах.

		вручную "стучаться", что он жив) .	
Lease Duration (Срок аренды)	Как часто издатель должен подтверждать свою активность (для Liveliness).	Длительность (Duration).	В паре с Liveliness.

Стандартные профили QoS

Выставлять каждую из 8 политик вручную — мучительно. Поэтому в ROS 2 есть предустановленные профили для типовых задач.

Профиль	History	Depth	Reliability	Durability	Для чего
`Default` (по умолчанию)	Keep last	10	Reliable	Volatile	Компромиссный вариант, похожий на поведение ROS 1. Подходит для большинства алгоритмов.
`Sensor Data`	Keep last	Небольшая (5)	Best Effort	Volatile	Для данных с сенсоров (камеры, лидары). Важна скорость, потеря нескольких сканов допустима.
`Parameters`	Keep last	1000	Reliable	Volatile	Для сервисов параметров, чтобы запросы не терялись.
`Services`	Keep last	10	Reliable	Volatile	Для сервисов, где важна надёжность, но не нужно лэтчинг.
`System Default`	Зависит от реализации DDS.				Использует настройки конкретного DDS (Fast DDS, Cyclone DDS и т.д.).

Важно: Профиль `Default` для **издателей** в ROS 2 (по умолчанию `Reliable`) — это осознанный выбор разработчиков, чтобы упростить переход с ROS 1. Однако, если вы создаёте подписчик с профилем `Sensor Data` (`Best Effort`), а издатель по умолчанию — `Reliable`, они **не увидят друг друга** (см. раздел о совместимости ниже).

Совместимость QoS (почему данные могут "не ехать")

Это самая частая причина проблем в ROS 2. **Подписчик и издатель соединяются, только если их профили QoS совместимы.**

Как это работает?

- **Подписчик (Subscription)** запрашивает профиль — минимальное качество, которое он готов принять.
- **Издатель (Publisher)** предлагает профиль — максимальное качество, которое он может обеспечить.
- Соединение происходит, если запрошенное качество не выше предложенного.

Таблицы совместимости для ключевых политик

Reliability

Издатель (Offers) ↓ \ Подписчик (Requests) →	Best Effort	Reliable
	Best Effort	Reliable
Best Effort	✓ Да	✗ Нет
Reliable	✓ Да	✓ Да

- **Суть:** Надёжный издатель (Reliable) может работать с ненадёжным подписчиком (Best Effort), но соединение будет работать в режиме Best Effort. А вот наоборот — никогда.

Durability

Издатель (Offers) ↓ \ Подписчик (Requests) →	Volatile	Transient Local
	Volatile	Transient Local
Volatile	✓ Да (только новые сообщения)	✗ Нет
Transient Local	✓ Да (только новые сообщения)	✓ Да (новые + "исторические")

- **Суть:** Чтобы "поздний" подписчик получил историю (Transient Local), и издатель, и подписчик должны использовать Transient Local. Это как раз настройка для статических publisher'ов.

Deadline u Lease Duration

Здесь правило простое: подписчик может запросить дедлайн больше или равный тому, что предлагает издатель.

Что происходит при несовместимости?

Ничего страшного не происходит — просто данные не передаются. И, что самое коварное, **явной ошибки по умолчанию нет**. В `rqt_graph` вы увидите, что ноды соединены, но сообщения не идут. Именно это часто приводит в бешенство новичков.

QoS-события: Узнаём о проблемах

Чтобы не гадать, почему данные не едут, можно подписаться на **QoS-события**. Для каждого издателя и подписчика можно задать callback, который сработает при наступлении события:

Для издателя (Publisher):

- `offered_deadline_missed`: Издатель пропустил свой дедлайн.
- `liveliness_lost`: Издатель потерял активность.
- `offered_incompatible_qos`: Издатель обнаружил подписчика с несовместимым QoS.

Для подписчика (Subscription):

- `requested_deadline_missed`: Подписчик не получил сообщение в ожидаемый дедлайн.
- `liveliness_changed`: Какой-то издатель на этом топике потерял/восстановил активность.
- `requested_incompatible_qos`: Подписчик обнаружил издателя с несовместимым QoS.

Это мощнейший инструмент отладки, позволяющий системе самой сигнализировать о проблемах коммуникации.

Работа с QoS в коде (Python примеры)

В Python всё делается через модуль `rclpy.qos`. Импорты и синтаксис немного отличаются, но логика та же.

Импорты

```
from rclpy.qos import QoSProfile, QoSDurabilityPolicy,
QoSHistoryPolicy, QoSReliabilityPolicy
from rclpy.qos import QoSDepth, KeepLast, KeepAll
from rclpy.duration import Duration
```

Явное создание профиля

```
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, QoSReliabilityPolicy,
QoSDurabilityPolicy, QoSHistoryPolicy
from rclpy.duration import Duration

# Создаём профиль "под себя"
custom_qos_profile = QoSProfile(
    history=QoSHistoryPolicy.KEEP_LAST,    # Keep last
    depth=5,                               # Depth = 5
    reliability=QoSReliabilityPolicy.RELIABLE, # Reliable
    durability=QoSDurabilityPolicy.TRANSIENT_LOCAL, #
    Transient Local
    deadline=Duration(seconds=0, nanoseconds=50_000_000), #
    50 мс
    lifespan=Duration(seconds=1)           # 1 секунда
)

# Используем профиль при создании publisher'a
publisher = self.create_publisher(
    MsgType,
    'topic_name',
    custom_qos_profile
)
```

Использование стандартных профилей

Самый простой и правильный способ — использовать встроенные профили, которые возвращают готовые объекты `QoSProfile`.

```
# Для сенсорных данных (Best Effort, малая глубина)
from rclpy.qos import SensorDataQoS

sensor_pub = self.create_publisher(
    MsgType,
    'sensor_data',
    SensorDataQoS()
)

# Для обычного publisher'a (Reliable, Depth 10)
from rclpy.qos import qos_profile_system_default # или
qos_profile_default
# rclpy по умолчанию: KeepLast, depth=10, Reliable, Volatile
default_pub = self.create_publisher(
    MsgType,
    'regular_data',
    qos_profile_default
)
```

```

# Для параметров (большая глубина)
from rclpy.qos import qos_profile_parameters

param_pub = self.create_publisher(
    MessageType,
    'param_data',
    qos_profile_parameters
)

# Для сервисов
from rclpy.qos import qos_profile_services_default

service = self.create_service(
    SrvType,
    'service_name',
    callback,
    qos_profile=qos_profile_services_default
)

```

Примечание: В Python есть несколько predefined профилей: `qos_profile_default`, `qos_profile_sensor_data`, `qos_profile_parameters`, `qos_profile_services_default`, `qos_profile_action_status`. Они находятся в модуле `rclpy.qos`.

Создание "статического" publisher'a (как *latching* в ROS 1)

Чтобы поздно подключившийся подписчик получил последнее сообщение, нужно использовать `transient_local` durability.

```

from rclpy.qos import QoSProfile, QoSReliabilityPolicy,
QoSDurabilityPolicy, QoSHistoryPolicy

# Publisher с глубиной 1 и transient_local
latching_qos = QoSProfile(
    history=QoSHistoryPolicy.KEEP_LAST,
    depth=1,
    reliability=QoSReliabilityPolicy.RELIABLE,
    durability=QoSDurabilityPolicy.TRANSIENT_LOCAL
)

latching_pub = self.create_publisher(
    MessageType,
    'latched_topic',
    latching_qos
)

```

Подписка на QoS-события (Python)

В Python события настраиваются через аргументы `event_callbacks` при создании подписчика или publisher'a.

```

# Для подписчика
def deadline_callback(event):
    self.get_logger().warn('Deadline missed!')

subscription = self.create_subscription(
    MessageType,
    'topic',
    callback,
    qos_profile,
    event_callbacks=rclpy.qos.EventCallbacks(
        deadline=deadline_callback
    )
)

# Для издателя
def incompatible_qos_callback(event):
    self.get_logger().error('Incompatible QoS!')

publisher = self.create_publisher(
    MessageType,
    'topic',
    qos_profile,
    event_callbacks=rclpy.qos.EventCallbacks(
        incompatible_qos=incompatible_qos_callback
    )
)

```

Диагностика: Как посмотреть QoS работающей системы

1. `ros2 topic info /topic_name --verbose`

Это твой главный инструмент. Он покажет реальные профили QoS издателя и всех подключённых подписчиков.

```
ros2 topic info /your_topic --verbose
```

Вывод покажет что-то вроде:

```

Publisher:
  History: Keep last
  Depth: 10
  Reliability: Reliable
  Durability: Volatile
  ...
Subscription:
  - History: Keep last
    Depth: 5
    Reliability: Best Effort
    Durability: Volatile

```

...

2. Отладка совместимости

Если после проверки вы видите, что издатель **Reliable**, а подписчик **Best Effort** — соединение будет. Если наоборот (**Best Effort** издатель и **Reliable** подписчик) — соединения **не будет**. Эта асимметрия — ключ к пониманию многих проблем.

🔗 Продвинутые темы и нюансы

JSON-представление QoS

В некоторых инструментах (например, в симуляторе NVIDIA Isaac Sim) QoS можно задавать в виде JSON-строки. Это удобно для конфигурации.

```
{
  "history": "keepLast",
  "depth": 10,
  "reliability": "reliable",
  "durability": "volatile",
  "deadline": 0.0,
  "lifespan": 0.0,
  "liveliness": "systemDefault",
  "leaseDuration": 0.0
}
```

📝 Финальная шпаргалка: Ты хочешь... Тебе нужно...

Ты хочешь...	Что использовать	Политика / Профиль
Гарантированную доставку всех команд управления	Reliability = Reliable	Профиль Default или Services
Максимальную скорость для данных с камеры/лидара	Reliability = Best Effort	Профиль Sensor Data
Чтобы поздно подключившийся подписчик получил последнее сообщение (например, карту)	Durability = Transient Local	Кастомный профиль с transient_local() и небольшой depth
Не потерять ни	History = Keep all	Кастомный профиль

одного сообщения, даже если подписчик "тормозит"		с <code>keep_all()</code> (осторожно, ресурсы!)
Понять, почему данные не идут, хотя топик виден	Проверить совместимость QoS	<code>ros2 topic info /topic -v</code> и таблицы выше
Узнать, что издатель "умер"	<code>Liveliness</code> + <code>Lease Duration</code> + события <code>liveliness_lost/changed</code>	Только для распределённых систем

Обратная связь

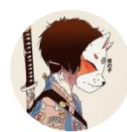
Если Вы нашли ошибку, неточность, у Вас есть предложения по улучшению или вопросы, напишите в Telegram Александру (https://t.me/Alex_19846) или Алисе (https://t.me/Kika_01). QR-коды со ссылками на их аккаунты представлены ниже.

Александр



@ALEX_19846

Алиса



@KIKA_01